MINI OPERATIONS ERP

Complete Project Documentation

Production-oriented full-stack Operations ERP

Documentation scope: requirements, architecture, database, APIs, business rules, security, frontend, testing, deployment, verification and release readiness.

Document	Value
Project	Mini Operations ERP
Implementation phases covered	Phase 2 through Phase 9
Backend	Node.js / Express / TypeScript / Prisma / PostgreSQL
Frontend	React 18 / TypeScript / Vite / Tailwind CSS / React Router / Axios
API documentation	OpenAPI 3.0 / Swagger UI at /api-docs
Automated regression	115 / 115 active tests passed
Final verified commit	76a45dd9db382bfcecfa0c6860be6d87b38959ba

Prepared from the supplied Full-Stack Developer Technical Case Study and the implementation / verification records for Phases 2–9.

Table of Contents

1. Executive Summary

Mini Operations ERP is a production-oriented full-stack operations system designed around a multi-location inventory workflow. The supplied case study explicitly evaluates the ability to connect a frontend, backend APIs, relational database, authentication, business logic, transactions, validation, testing and portable architecture.

The implemented system covers the requested operational flow: Inventory → Work Order → Stock Check → Internal Transfer / Shortage → Customer Reservation. The final implementation also includes automated regression coverage, OpenAPI documentation, database migration verification, security hardening and a responsive React application.

The implementation was delivered incrementally rather than as one final code drop. Phases 2–9 introduced and verified authentication/RBAC, inventory, work orders, transfers, customer orders, the production frontend, edge-case tests, API documentation and release-readiness checks.

Final verification recorded 115/115 active backend tests passing across seven suites, zero TypeScript errors, successful backend and frontend production builds, a clean Git working tree, and a documented OpenAPI surface containing 19 endpoints.

Important source boundary: the Technical Case Study defines the assignment requirements; the phase reports supplied in this conversation document what was implemented and verified. Where the case study does not specify an implementation detail, this document labels it as implementation evidence rather than a case-study requirement.

Case-study basis: the objective is a small but production-oriented full-stack Operations ERP, explicitly not a UI-only assignment. filecite turn4file0 L12-L25

2. Case Study Requirements and Scope

The case study defines five mandatory functional areas: Authentication & Roles, Inventory Management, Work Order + Material Stock Check, Internal Stock Transfer, and Customer Order & Stock Reservation.

Area	Case-study requirement	Implemented status
Authentication & Roles	Admin, Operations User, Sales User; backend authorization mandatory.	Implemented and regression-tested.
Inventory	Item, Category, Location, Batch, Physical, Reserved, Available; prevent negative/invalid/duplicate/reservation-overrun conditions.	Implemented with transactional mutation and audit behavior.
Work Orders	Admin creation; location, item, required quantity, assigned user, status; Assigned → In Progress → Completed; automatic shortage calculation.	Implemented with live shortage calculation.
Transfers	Requested → Dispatched → Received; source decreases on dispatch; destination increases only on receipt; duplicate receipt prevented.	Implemented with atomic dispatch/receipt logic.
Customer Orders	Sales creates order and reserves stock; concurrent requests cannot over-reserve.	Implemented with atomic PostgreSQL conditional updates.

Technical requirements from the case study include a frontend connected to backend APIs, relational data relationships, backend validation, error handling, authentication, RBAC, database transactions where required, and environment-based configuration. □filecite□turn4file0□L103-L114□

Minimum screens required by the case study are Login, Inventory, Work Orders, Internal Transfers and Customer Orders. The assignment explicitly prioritizes functionality over excessive UI design. □filecite□turn4file2□L242-L249□

Mandatory minimum tests are: cannot reserve more than available inventory; cannot transfer more than available inventory; destination stock increases only after transfer receipt; the same transfer cannot be received twice; and unauthorized users cannot perform restricted operations. □filecite□turn4file2□L250-L261□

3. System Architecture

The implementation follows a layered backend design reported during Phase 6: Routes → Controllers → Services → Repositories → Prisma, with PostgreSQL providing relational persistence. The frontend communicates with the backend through a centralized Axios client and JWT authorization headers.

High-level flow

Browser / React UI → Axios API client → Express routes → authentication / authorization middleware → controllers → services → repositories / Prisma → PostgreSQL

Business workflows are intentionally enforced at the backend/database level. The frontend role-aware controls improve usability, but they do not replace backend authorization.

Layer	Responsibility
Frontend	Screens, forms, role-aware navigation, loading/error/empty states, API interaction and responsive UI.
Routes	HTTP endpoint definitions and route-level middleware composition.
Authentication / RBAC	JWT validation and role authorization.
Controllers	HTTP request/response orchestration and validation handoff.
Services	Business rules, state transitions, transaction orchestration and allocation logic.
Repositories / Prisma	Persistence access and relational queries/updates.
PostgreSQL	Transactional source of truth for inventory, orders, transfers, work orders and audit records.

The case study requires a portable architecture and environment-based configuration so business logic is not heavily coupled to a single hosting provider or environment. □filecite□turn4file2□L239-L241□

4. Technology Stack and Project Structure

Layer	Technology / approach	Purpose
Frontend	React 18 + TypeScript	Application UI and typed component architecture.
Build / dev	Vite	Fast local development and production bundling.
Styling	Tailwind CSS	Responsive UI styling.
Routing	React Router v6	Protected application routes and navigation.
HTTP	Axios	Centralized API client and JWT header attachment.
Icons / UI	Lucide Icons + component-based UI	Consistent interface controls.
Backend	Node.js + Express + TypeScript	REST API and server-side business logic.
ORM	Prisma	Typed relational data access and transactions.
Database	PostgreSQL	Transactional relational persistence.
Auth	JWT + bcryptjs	Session authentication and password hashing.
Testing	Vitest + Supertest + Prisma + PostgreSQL	Integration, business-rule and concurrency verification.
API docs	OpenAPI 3.0 / Swagger UI	Interactive endpoint documentation.

Core backend test suites recorded in the final Phase 8/9 verification are auth.test.ts, inventory.test.ts, workOrder.test.ts, transfer.test.ts, customerOrder.test.ts, erp.test.ts and edgeCases.test.ts.

The repository history records separate feature commits for major phases, satisfying the case study's requirement for reasonable development history rather than a single final commit. The case study explicitly states not to submit the whole application as one final commit. □filecite□turn4file2□L281-L283□

5. Environment, Setup and Runbook

The exact environment-variable names should be taken from the repository's current .env.example / configuration files. The case study requires documentation of environment variables and environment-based configuration; this document therefore avoids inventing variable names that were not supplied in the project record.

Backend setup

```
cd backend
npm install
npx prisma validate
npx prisma migrate status
npx tsc --noEmit
npm run build
npm test
```

Frontend setup

```
cd frontend
npm install
npx tsc --noEmit
npm run build
npm run dev
```

Recorded local services

- Frontend development server: http://localhost:3000/ (as recorded during Phase 7).
- Backend API health endpoint: http://localhost:5000/api/health (as recorded during Phase 7).
- Swagger UI: http://localhost:5000/api-docs (as recorded during Phase 9).

Database verification recorded in Phase 9: `npx prisma validate` reported a valid schema, and `npx prisma migrate status` reported the database schema as up to date with one migration applied.

Case-study submission requirements explicitly include tech stack, project setup, database setup, environment variables, how to run and how to test in the README. □filecite□turn4file2□L262-L276□

6. Authentication and Role-Based Authorization

Authentication is centralized through JWT. The Phase 7 implementation uses AuthContext.tsx for token persistence, login state, quick demo role logins, session-expiration cleanup and logout. The Axios client automatically attaches `Authorization: Bearer <JWT>` to outgoing API requests.

Passwords are hashed with bcryptjs using 10 rounds, and password hashes are excluded from API responses. JWT claims include the authenticated subject and role; server-side authorization uses the authenticated context rather than trusting client-provided user identifiers.

Capability	ADMIN	OPERATIONS	SALES
Inventory read	Yes	Yes	Yes
Inventory adjustment	Yes	Yes	No
Work Order create	Yes	No	No
Work Order view	Yes	Yes	Yes
Work Order status update	Yes	Yes	No
Transfer create / dispatch / receive	Yes	Yes	No
Customer Order create / reserve	Yes	No	Yes
Customer Order view	Yes	Yes	Yes
Customer Order cancel / release	Yes	No	Yes

The case study requires backend authorization, with the intended examples that Admin can create Work Orders, Operations manages inventory/transfers, and Sales creates orders/reserves stock. □filecite□turn4file0□L31-L41□

7. Inventory Management

Inventory records are modeled around item, category, location and batch, with physical, reserved and available quantities. The implemented invariant is `availableQuantity = physicalQuantity - reservedQuantity`. Inventory mutation is performed through backend services and produces audit records.

Core invariants

- Available quantity cannot become negative.
- Invalid quantities are rejected.
- Reservations cannot exceed available quantity.
- Duplicate inventory mutations are guarded through idempotency mechanisms.
- Reservation changes physical stock by zero: physical remains unchanged while reserved increases and available decreases.
- Release/cancellation reverses reservation: reserved decreases, available increases, physical remains unchanged.

Stock adjustment

ADMIN and OPERATIONS may perform physical stock adjustments. The frontend exposes an adjustment modal with an audit reason. The backend applies validation and records inventory transactions.

The case study explicitly requires prevention of negative inventory, invalid quantity, duplicate inventory transactions and reservation beyond available quantity. □filecite□turn4file0□L42-L57□

8. Work Orders and Dynamic Shortage Engine

Work Orders represent material requirements at a location. A Work Order does not reserve or consume stock. Instead, shortage is calculated dynamically whenever the Work Order is read.

Dynamic shortage formula

```
shortage = max(0, requiredQuantity - currentAvailableQuantity)

currentAvailableQuantity =
    sum(availableQuantity)
    across inventory batches matching:
      itemId + locationId
```

This design deliberately avoids persisting shortage as a mutable Work Order field. Consequently, inventory changes immediately affect the reported shortage without updating the Work Order row.

Lifecycle	Allowed transition
Initial	ASSIGNED
ASSIGNED	IN_PROGRESS
IN_PROGRESS	COMPLETED
COMPLETED	Backward transitions rejected with HTTP 400

The Phase 4 verification demonstrated: required 60 with available 20 reports shortage 40; increasing available stock to 60 makes shortage 0; reducing available stock to 30 makes shortage 30. Work Order creation does not mutate inventory.

The case study requires Admin Work Order creation, the minimum fields Work Order ID, location, item, required quantity, assigned user and status, and the Assigned → In Progress → Completed lifecycle with automatic shortage calculation. □filecite□turn4file0□L58-L73□

9. Internal Stock Transfers

Transfers move stock between locations through an explicit lifecycle: REQUESTED → DISPATCHED → RECEIVED.

Operation	Inventory effect	Protection
Create REQUESTED	No inventory change.	Requester identity taken from JWT.
Dispatch	Source available/physical stock decreases; destination unchanged.	Atomic conditional update requires sufficient available stock.
Receive	Destination physical/available stock increases; reserved quantity is preserved.	Only DISPATCHED transfers can be received; duplicate receive blocked.

Dispatch is performed transactionally using a conditional database update equivalent to `WHERE availableQuantity >= quantity`. This means parallel dispatch attempts cannot both deduct the same limited stock. Phase 5 verified exactly one success and one failure for the insufficient-stock race scenario.

Receipt preserves the composite item + destination location + batch identity. Cross-item batch assignments are rejected. If the destination inventory row is missing, the implementation safely creates the matching row with reserved quantity zero.

The case study's required rule is explicit: source inventory reduces on dispatch, destination must not increase before receipt, destination increases on receipt, and the same transfer cannot be received twice. filecite turn4file0 L74-L94

10. Customer Orders and Atomic Stock Reservation

Customer Orders reserve inventory for fulfillment without physically consuming stock. Reservation is an atomic database operation inside a transaction.

Reservation invariant

```
reservedQuantity = reservedQuantity + quantity
availableQuantity = availableQuantity - quantity
physicalQuantity = unchanged

WHERE inventory.id = targetInventoryId
  AND availableQuantity >= quantity
```

If the conditional update affects zero rows, the reservation cannot proceed and the transaction rolls back. Multi-batch allocation can consume available stock from multiple matching batches deterministically; if the total is insufficient, the complete operation is rolled back.

Cancellation changes the order status to CANCELLED and releases reserved stock: reserved decreases, available increases, physical remains unchanged. A RELEASE audit transaction is created. Duplicate cancellation is rejected using an idempotency check.

Concurrency verification recorded in Phases 6 and 8: with available stock of 100, parallel reservations of 80 and 50 produce exactly one HTTP 201 success and one HTTP 400 failure; final reserved stock is never 130.

Cross-module interaction: a Work Order requiring 60 at a location with available 100 initially has shortage 0. Reserving 80 reduces available to 20, so the Work Order immediately reports shortage 40. Cancelling the order restores available to 100 and shortage to 0.

The case study requires that concurrent users must not reserve more stock than actually exists and explicitly requires the problem to be solved at backend/database level. □filecite□turn4file0□L95-L102□

11. Frontend Application and User Experience

Phase 7 delivered a fresh React application connected to the existing backend. The application includes protected routing, centralized authentication, responsive navigation and role-aware actions.

Screen / route	Purpose	Role behavior
Login /login	JWT login, quick demo role login and session entry.	All roles.
Dashboard /dashboard	Executive overview and operational KPIs.	Authenticated users.
Inventory /inventory	Physical/reserved/available stock and adjustments.	Read: all; adjustment: Admin/Ops.
Work Orders /work-orders	Material requirements, live shortage and lifecycle status.	Create: Admin; status: Admin/Ops; view: all.
Transfers /transfers	Requested → dispatched → received stock movement.	Mutations: Admin/Ops; view: all.
Customer Orders /orders	Reservations, order details and cancellation/release.	Create/cancel: Admin/Sales; view: all.

Phase 7 also introduced an AppShell/Sidebar/StatCard-oriented visual system and a minimalist black/white/ocean-blue design direction. Subsequent UI work was requested to add smooth transitions, scrolling animations and refined selection controls while preserving backend behavior.

Important: frontend restrictions are convenience and UX controls only. The backend remains the authority for authorization and business rules.

12. API Documentation — Complete Endpoint Inventory

Method	Endpoint	Auth / role	Purpose	Success	Errors
POST	/api/auth/login	Public	Login and obtain JWT	200	400, 401
GET	/api/auth/me	All authenticated	Current user profile	200	401
GET	/api/inventory	All authenticated	List inventory	200	401
GET	/api/inventory/:id	All authenticated	Inventory detail	200	401, 404
POST	/api/inventory/adjust	ADMIN, OPERATIONS	Adjust physical stock + audit	200	400, 401, 403, 409
POST	/api/work-orders	ADMIN	Create Work Order	201	400, 401, 403
GET	/api/work-orders	All authenticated	List Work Orders + live shortage	200	401
GET	/api/work-orders/:id	All authenticated	Work Order detail + live shortage	200	401, 404
PATCH	/api/work-orders/:id/status	ADMIN, OPERATIONS	Lifecycle status update	200	400, 401, 403
POST	/api/transfers	ADMIN, OPERATIONS	Create REQUESTED transfer	201	400, 401, 403, 404
GET	/api/transfers	All authenticated	List transfers	200	401
GET	/api/transfers/:id	All authenticated	Transfer detail	200	401, 404
POST	/api/transfers/:id/dispatch	ADMIN, OPERATIONS	Dispatch source stock	200	400, 401, 403
POST	/api/transfers/:id/receive	ADMIN, OPERATIONS	Receive destination stock	200	400, 401, 403
POST	/api/customer-orders	ADMIN, SALES	Create order + reserve	201	400, 401, 403, 404
GET	/api/customer-orders	All authenticated	List customer orders	200	401
GET	/api/customer-orders/:id	All authenticated	Customer order detail	200	401, 404
POST	/api/customer-orders/:id/cancel	ADMIN, SALES	Cancel + release	200	400, 401, 403, 404
GET	/api/health	Public	Health check	200	—

Swagger UI

The Phase 9 report records an interactive OpenAPI 3.0 / Swagger UI at `/api-docs`, with BearerAuth security documented at the API level. The reported API inventory contains 19 endpoints.

Example OpenAPI metadata

```
openapi: 3.0.0
info:
  title: Mini Operations ERP API
  version: 1.0.0
security:
  - BearerAuth: []
```

13. Database, Relationships and Integrity

The system uses PostgreSQL as its relational database with Prisma as the ORM. The case study requires a relational database with proper data relationships; Phase 9 recorded successful Prisma schema validation and migration-status verification.

Core domain entities evidenced by the implementation

Entity	Role in the domain
User	Authentication identity, role and authenticated actor for protected operations.
Item	Material/product identity used by inventory and operational records.
Category	Classification for inventory items.
Location	Physical operating site / inventory location.
Batch	Batch identity associated with an item and inventory movement.
Inventory	Stock at an item/location/batch combination, including physical/reserved/available quantities.
InventoryTransaction	Audit trail for adjustments, transfer dispatch/receipt and reservation release.
WorkOrder	Material requirement at a location with assigned user and lifecycle status.
StockTransfer	Internal movement request between source and destination locations.
CustomerOrder	Customer reservation record and its lifecycle.

Composite integrity

Phase 9 reports composite item/batch integrity using a unique composite relationship so inventory and transfer records cannot associate a batch with a different item. The reported schema also uses composite identity involving item, destination location and batch when receiving transfer stock.

Migration status

Recorded result: `npx prisma migrate status` reported the database schema as up to date with one migration applied.

The case study explicitly requires submission of a Database Schema / ER Diagram.
filecite turn4file2 L262-L276

14. Transactions, Concurrency and Idempotency

Transaction boundaries are central to correctness. The system uses database transactions for operations where partial completion would violate inventory integrity.

Workflow	Concurrency / transaction strategy
Customer reservation	Conditional atomic UPDATE requires availableQuantity >= requested quantity; whole order transaction rolls back on failure.
Multi-batch reservation	Allocates across batches inside one transaction; insufficient total stock rolls back all allocations.
Transfer dispatch	Atomic conditional source-stock update; parallel dispatches cannot both deduct insufficient stock.
Transfer receive	Transactional destination update with lifecycle and duplicate-receipt protection.
Order cancellation	Transactional release of reserved stock and cancellation state; duplicate cancellation blocked.
Inventory mutation	Audit transaction/idempotency safeguards prevent duplicate processing.

Idempotency keys reported by Phase 9 include inventory mutations, transfer dispatches, transfer receipts and customer-order cancellations. Examples recorded in the phase reports include `RECEIVE-${transferId}` and `RELEASE-${orderId}`.

Rollback guarantees

- Failed reservation does not leave partial stock reservations.
- Failed dispatch leaves transfer and inventory unchanged.
- Failed receive leaves transfer and destination inventory unchanged.
- Failed order creation or audit logging rolls back the surrounding reservation transaction.

15. Validation and Error Handling

The backend validates quantities, resource existence, lifecycle states, role permissions and cross-entity integrity before committing mutations.

Validation area	Representative behavior
Quantity	Positive integer requirements for Work Orders; invalid zero/float quantities rejected.
Resource existence	Unknown item, location or assigned user rejected with appropriate 404 behavior.
Status	Invalid Work Order or transfer lifecycle transitions rejected with HTTP 400.
Availability	Insufficient available stock rejected before reservation/dispatch can overdraw inventory.
Authorization	Restricted operations return HTTP 403 for disallowed roles.
Duplicate processing	Duplicate receipt/cancellation/mutation is rejected through state and idempotency safeguards.
Cross-item batch	Batch whose item does not match the transfer item is rejected.

Phase 9 reports a centralized Express error handler returning structured JSON errors in the form `{ success: false, error: "..." }`, without exposing raw Prisma stack traces or SQL statements.

16. Security Hardening

- Passwords are stored as bcryptjs hashes using 10 rounds; password hashes are excluded from API responses.
- JWTs are signed and validated for expiration and claims; user identity for protected mutations comes from authenticated server context.
- Client-side user-ID spoofing is rejected for fields such as createdById/requestedById/salesUserId because the backend assigns actor identity from JWT context.
- Backend RBAC is enforced independently of the frontend.
- Atomic conditional SQL prevents over-reservation and negative available stock during concurrency races.
- Idempotency safeguards prevent duplicate mutation processing.
- Centralized error handling sanitizes implementation details from API errors.
- Relational/composite integrity protects item-batch associations.

The case study states that AI tools may be used, but the candidate must understand everything submitted and may be asked to explain the database, modify an API, change a business rule, debug failures, add a field or fix a transaction issue. filecite turn4file2 L284-L295

17. Testing and Verification

The project substantially exceeds the five minimum tests specified by the case study. Final Phase 8/9 verification reports 115/115 active tests passing across seven suites.

Suite	Tests	Coverage
auth.test.ts	17	JWT authentication, password hashing and role-token verification.
inventory.test.ts	17	Inventory listing, invariants, adjustments and mutation behavior.
workOrder.test.ts	19	Work Order creation, validation, dynamic shortage and lifecycle.
transfer.test.ts	23	Transfer lifecycle, atomic dispatch, receipt and duplicate protection.
customerOrder.test.ts	22	Reservation, multi-batch allocation, concurrency and release.
erp.test.ts	6	Core case-study business-rule assertions.
edgeCases.test.ts	11	RBAC matrix, JWT security and race/edge conditions.
TOTAL	115	100% pass rate in the recorded final run.

Recorded final command

```
npx vitest run "src/__tests__/auth.test.ts" "src/__tests__/inventory.test.ts" "src/__tests__/workOrder.test.ts" "src/__tests__
```

Final recorded test result: 7 test files passed, 115 tests passed, 0 failed, duration approximately 12.69 seconds.

Additional verification

- Backend `npx tsc --noEmit`: 0 errors.
- Backend `npm run build`: success.
- Frontend `npx tsc --noEmit`: 0 errors.
- Frontend `npm run build`: success; production dist generated.
- Prisma schema validation: success.
- Prisma migration status: schema up to date.
- Git working tree: clean at the recorded final Phase 6/7/9 checkpoints.

18. Phase-by-Phase Delivery History

Phase	Delivered	Recorded verification
Phase 2	Authentication, JWT, password hashing and RBAC.	17/17 auth tests.
Phase 3	Inventory engine, invariants, adjustments and audit behavior.	17/17 inventory tests.
Phase 4	Work Orders, lifecycle and dynamic shortage engine.	19/19 Work Order tests.
Phase 5	Internal Stock Transfer lifecycle and atomic dispatch/receipt.	23/23 transfer tests; concurrency verified.
Phase 6	Customer Orders and concurrency-safe atomic reservation.	22/22 customer-order tests; reservation race verified.
Phase 7	Fresh production-oriented React frontend and role-aware UI.	Frontend TypeScript/build successful; backend regression retained.
Phase 8	Comprehensive edge-case/RBAC/concurrency regression suite.	115/115 total active tests.
Phase 9	Swagger/OpenAPI, security audit, DB/migration verification and release readiness.	API inventory 19 endpoints; builds and schema verification successful.

Representative commits recorded in the supplied phase reports include `0902329d` for Work Orders, `61404292` for Stock Transfers, `76a45dd` for Customer Orders, and `4ef75d7` for the fresh frontend. The Phase 6 report records the full commit `76a45dd9db382bfcecfa0c6860be6d87b38959ba`.

19. Deployment and Production Readiness

Production readiness in the supplied Phase 9 report is based on functional completeness, security review, schema verification, automated regression and successful production builds.

Release checklist

Check	Recorded result
API documentation	OpenAPI 3.0 / Swagger UI available at /api-docs.
Backend tests	115/115 active tests passed.
Backend TypeScript	0 errors.
Backend build	Successful.
Frontend TypeScript	0 errors.
Frontend build	Successful production bundle.
Database schema	Prisma validate successful.
Database migrations	Schema up to date; one migration applied.
Security	JWT/RBAC, hashing, sanitized errors, concurrency and idempotency reviewed.
Git history	Feature-oriented development history recorded.

The system was marked APPROVED & RELEASE READY in the supplied Phase 9 report. This assessment is an implementation record, not a claim that every external deployment environment has been independently exercised.

20. Submission Checklist and Demo Plan

The case study requires the following submission artifacts:

Requirement	Documentation / project evidence
Git repository	Complete source code with phase-oriented commit history.
README	Should contain tech stack, setup, database setup, environment variables, run and test instructions.
Database schema / ER diagram	Relational model and integrity explanation are documented; include the repository's actual ER diagram/schema export when submitting.
API documentation	Swagger/OpenAPI 3.0 at /api-docs; 19 endpoints documented in the project record.
Short demo video	5–7 minute walkthrough covering Login → Inventory → Work Order → Transfer → Order Reservation.

The source case study explicitly requires the demo sequence Login → Inventory → Work Order → Transfer → Order Reservation and a maximum length of 5–7 minutes. filecite turn4file2 L262-L280

Suggested demo sequence

- Login as Admin and demonstrate authenticated entry.
- Open Inventory and show physical/reserved/available values.
- Create a Work Order and show its shortage state.
- Use a transfer to move material; show that destination stock changes only after receipt.
- Log in as Sales and create a Customer Order reservation.
- Show reserved increasing and available decreasing while physical remains unchanged.
- Cancel the order and show reserved stock released.
- Optionally show Swagger UI and the test/build verification as closing evidence.

21. Engineering Notes and Live Verification Readiness

The case study includes a live-verification stage in which shortlisted candidates may receive an unannounced change. Examples include damaged stock reducing available stock, partially receiving a transfer, cancelling an order and releasing reserved inventory, or restricting users to assigned locations.

Potential live change	Relevant extension point
Damaged stock reduces available stock	Inventory model/business-rule and transaction logic.
Partial transfer receipt	Transfer lifecycle, quantity tracking and receipt transaction.
Order cancellation releases reservation	Customer Order cancellation/release transaction; already implemented in the current system.
Restrict users to assigned location	JWT user context, authorization middleware and location-aware service checks.

Because the case study says candidates may be asked to modify or explain their application, the final repository should be treated as an engineering system that the developer can reason about, debug and extend—not merely as a static demonstration.

The case study explicitly states that if the candidate cannot modify or explain the application, the submission will not qualify. □filecite□turn4file2□L284-L295□

22. Final Release Assessment

Based on the supplied phase verification reports, Mini Operations ERP has progressed from core backend capabilities through frontend delivery, comprehensive testing and final production hardening.

The final recorded state includes:

- All mandatory functional modules implemented.
- Backend-enforced authentication and RBAC.
- Relational PostgreSQL persistence with Prisma.
- Inventory invariants and audit behavior.
- Dynamic, read-time Work Order shortage calculation.
- Atomic transfer dispatch and receipt lifecycle.
- Atomic customer stock reservation with concurrency protection.
- Order cancellation with stock release.
- Duplicate processing safeguards.
- Responsive React frontend connected to backend APIs.
- OpenAPI/Swagger documentation.
- 115/115 active automated tests passing.
- Successful backend and frontend TypeScript checks and production builds.
- Successful Prisma schema/migration verification.

Release-readiness conclusion

The supplied Phase 9 verification report states that the system is APPROVED & RELEASE READY. This document preserves that conclusion as the project's recorded implementation assessment while distinguishing it from the original case-study requirements.

Source and evidence notes

Primary requirement source: Full-Stack Developer Technical Case Study — Mini Operations ERP, supplied in the conversation. The case study defines the mandatory modules, technical requirements, minimum screens, mandatory tests, submission requirements, AI-use expectations, live verification examples and evaluation criteria. □filecite□turn4file0□L12-L37□ □filecite□turn4file2□L250-L322□

Implementation evidence source: Phase 2–9 walkthroughs and final verification reports supplied in the conversation, including test counts, endpoint inventory, architecture notes, security checks, build results and Git checkpoints.

END OF DOCUMENT